

Skrypty łączenia z bazą

MySQL Improved

Biblioteka `mysqli` (MySQL Improved) jest standardem w PHP do komunikacji z bazami danych MySQL. Podejście proceduralne opiera się na wywoływaniu gotowych funkcji, którym **jako pierwszy** argument zazwyczaj przekazujemy identyfikator połączenia.

MySQL Improved

Główne zalety:

- Prostota dla początkujących.
- Bezpośrednie odwzorowanie kroków: Połącz -> Zapytaj -> Pobierz dane -> Zamknij.
- Pełna kompatybilność z wymaganiami egzaminacyjnymi.

Zestaw podstawowych funkcji

Tabela 1. Wybór funkcji języka PHP do obsługi bazy MySQL i MariaDB

Funkcje biblioteki mysqli	Zwracana wartość
<code>mysqli_connect(serwer, użytkownik, hasło, nazwa_bazy)</code>	id połączenia lub FALSE, gdy niepowodzenie
<code>mysqli_select_db(id_polaczenia, nazwa_bazy)</code>	TRUE/FALSE w zależności od stanu operacji
<code>mysqli_error(id_polaczenia)</code>	Tekst komunikatu błędu
<code>mysqli_close(id_polaczenia)</code>	TRUE/FALSE w zależności od stanu operacji
<code>mysqli_query(id_polaczenia, zapytanie)</code>	Wynik zapytania
<code>mysqli_fetch_row(wynik_zapytania)</code>	Tablica numeryczna odpowiadająca wierszowi zapytania
<code>mysqli_fetch_array(wynik_zapytania)</code>	Tablica zawierająca kolejny wiersz z podanych w wyniku zapytania lub FALSE, jeżeli nie ma więcej wierszy w wyniku zapytania
<code>mysqli_num_rows(wynik_zapytania)</code>	Liczba wierszy w podanym zapytaniu
<code>mysqli_num_fields(wynik_zapytania)</code>	Liczba kolumn w podanym zapytaniu
<code>isset(\$zmienna)</code>	TRUE/FALSE w zależności od tego, czy \$zmienna istnieje

Przykład

```
<?php
$idPolaczenia = mysqli_connect("localhost", "root","", "galeria") ;
if ($idPolaczenia){
    //$sql = "SELECT tytul, autor FROM obrazy;";
    $sql = "SELECT zdjecie FROM obrazy;";
    $wynikZapytania=mysqli_query($idPolaczenia,$sql);
    while ( $wierszTablica = mysqli_fetch_row($wynikZapytania)){
        echo "<img src='$wierszTablica[0]'>";
    }
}

mysqli_close($idPolaczenia);
}
?>
```

Nawiązanie połączenia – krok 1

Argumentami funkcji nawiązującej połączenie muszą być stringi. Nie można opuścić żadnego argumentu.

```
$idPolaczenia = mysqli_connect("localhost", "root","", "galeria") ;  
if ($idPolaczenia){
```

Rozłączenie, zamknięcie połączenia – krok 4

```
|  
mysqli_close($idPolaczenia);  
}
```

Wysłanie zapytania – krok 2

Najpierw należy sformułować zapytanie, po czym wygodnie jest zapisać go do zmiennej. Należy pamiętać, że musi ono być typu string. Potem używa się funkcji wysyłającej zapytanie do bazy (identyfikatora połączenia)

✓ Pokaż jako kod PHP

```
$sql = "SELECT zdjecie, id FROM obrazy;";
```

<pre>mysqli_query(id_polaczenia, zapytanie)</pre>	Wynik zapytania
---	-----------------

Wysłanie zapytania – krok 2

Zapytanie jest wysyłane jeśli mamy połączenie z bazą:

```
if ($idPolaczenia){  
    //$sql = "SELECT tytul, autor FROM obrazy;";  
    $sql = "SELECT zdjecie FROM obrazy;";  
    $wynikZapytania=mysqli_query($idPolaczenia,$sql);
```

Pobieranie danych – krok 3

Otrzymany wynik zapytania zawiera pewną liczbę rekordów – każdy rekord to jeden wiersz w tabeli. Dostęp do rekordów jest sekwencyjny. Najpierw uzyskujemy dostęp do pierwszego, potem do drugiego itd. Nie można wracać do poprzednich rekordów!

	▼	id	tytuł	autor	cena	zdjecie
⌵	Delete	1	Tęsknota	Alina Bielaj	5300	brzozy.jpg
⌵	Delete	2	Do domu	Alina Bielaj	6100	droga.jpg
⌵	Delete	3	Popołudnie	Alina Bielaj	5900	widok.jpg
⌵	Delete	4	Jesień	Alina Bielaj	4900	drzewo.jpg
⌵	Delete	5	Imieniny	Zuzanna Wiech	4500	bukiet.jpg
⌵	Delete	6	Lato	Zuzanna Wiech	4200	dmuchawiec.jpg
⌵	Delete	7	Niewinność	Zuzanna Wiech	5600	kwiat.jpg
⌵	Delete	8	Ulotność	Zuzanna Wiech	4800	motyl.jpg
⌵	Delete	9	Zima - Styczen	Piotr Milicki	3500	styczen.jpg
⌵	Delete	10	Zima - Luty	Piotr Milicki	5100	luty.jpg
⌵	Delete	11	Jesien - Wrzesien	Piotr Milicki	4900	wrzesien.jpg
⌵	Delete	12	Jesien - Pazdziernik	Piotr Milicki	4900	pazdziernik.jpg
⌵	Delete	13	Poranna mgła	Krystyna Zych	3500	poranek.jpg
⌵	Delete	14	Świat lodem skutý	Krystyna Zych	4200	zima.jpg

Pobieranie danych – krok 3

Stosując się do informacji z poprzedniego slajdu, będziemy pobierać wiersz po wierszu w pętli. Pętla będzie działać tak długo, jak długo daje się pobierać kolejny rekord. Naturalnym wyborem zatem pętla while.

Popatrzmy jeszcze na pojedynczy rekord:

id	tytuł	autor	cena	zdjęcie
1	Tęsknota	Alina Bielaj	5300	brzozy.jpg

Naturalną strukturą do przechowywania takich danych jest tablica.

Pobieranie danych – krok 3

Pobierany rekord trafia zatem do tablicy. Może to być tablica asocjacyjna lub indeksowana, a wyboru jej rodzaju dokonuje programista kierując się własnym gustem. W zależności od rodzaju tablicy wybieramy jedną z funkcji

<code>mysqli_fetch_row(wynik_zapytania)</code>	Tablica numeryczna odpowiadająca wierszowi zapytania
<code>mysqli_fetch_array(wynik_zapytania)</code>	Tablica asocjacyjna odpowiadająca wierszowi zapytania lub FALSE, jeżeli nie ma więcej wierszy w wyniku zapytania

Pobieranie danych – krok 3

Porównanie dla lepszego zrozumienia - zestawienie różnic w dostępie do danych (zakładając, że pierwsza kolumna w bazie to "imie"):

- `mysqli_fetch_row` : Dostęp przez indeks liczbowy: `$row[0]`
- `mysqli_fetch_assoc` : Dostęp przez nazwę kolumny: `$row['imie']`
- `mysqli_fetch_array` : Dostęp na oba sposoby: `$row[0]` LUB `$row['imie']` (domyślnie tworzy podwójną tablicę).

Pobieranie danych – krok 3

I w kodzie :

```
while ( $wierszTablica = mysqli_fetch_row($wynikZapytania)){  
    echo "<img src='$wierszTablica[0]'>";  
    // echo "Autor: ".$wierszTablica[1];  
    // echo "    tytuł: ".$wierszTablica[0];  
    // echo "<br>";  
}
```

Cały skrypt

```
<?php
$idPolaczenia = mysqli_connect("localhost", "root","", "galeria") ;
if ($idPolaczenia){
    //$sql = "SELECT tytul, autor FROM obrazy;";
    $sql = "SELECT zdjecie FROM obrazy;";
    $wynikZapytania=mysqli_query($idPolaczenia,$sql);
    while ( $wierszTablica = mysqli_fetch_row($wynikZapytania)){
        echo "<img src='$wierszTablica[0]'">";
        // echo "Autor: ".$wierszTablica[1];
        // echo "    tytuł: ".$wierszTablica[0];
        // echo "<br>";
    }
    mysqli_close($idPolaczenia);
}
?>
```

Wyświetlanie rekordów

W pętli decydujemy jakie pola z rekordów i w jaki sposób chcemy wyświetlić. Warto pamiętać, że łatwiej będzie jeśli maksymalnie dużo „wyrzucimy” do HTML’a. Przykładowo, jeśli mamy pokazać dane w tabelce to znaczniki `<table>` i `</table>` dajemy przed i po skrypcie. Podobnie będzie ze znacznikami list `ol` i `ul`. Do skryptu wrzucamy tylko `li`

Przykład

Egzamin z czerwca 2023 – zestaw 2, dane pobrane z bazy należało wrzucić do tabelki:

Hurtownia z najlepszymi cenami

Nasze ceny

Zeszyt 60 kartek	4.5
Zeszyt 32 kartki	1.2
Cyrkiel	12.4
Linijka 30 cm	7.2

Koszt zakupów

wybierz artykuł:

liczba sztuk:

wartość zakupów: 2.4

Kontakt



e-mail: hurt@poczta2.pl

Witrynę wykonał: 000000000000

Zeszyt 60 kartek	4.5
Zeszyt 32 kartki	1.2
Cyrkiel	12.4
Linijka 30 cm	7.2

Wyświetlanie rekordów

```
if ($wynikZapytania=mysqli_query($idPolaczenia, $sql)){  
    while ($wiersz=mysqli_num_rows($wynikZapytania)){  
        echo '<tr>';  
        echo "<td> $wiersz[0] </td>";  
        echo "<td> $wiersz[1] </td>";  
        echo '</tr>';  
    }  
}
```

Wyświetlanie rekordów

W świecie PHP wybór między cudzysłowem (") a apostrofem (') to coś więcej niż tylko kwestia estetyki. Choć oba służą do definiowania ciągów znaków (stringów), działają pod maską w zupełnie inny sposób. Patrz w kodzie:

```
if ($wynikZapytania=mysqli_query($idPolaczenia, $sql)){  
    while ($wiersz=mysqli_num_rows($wynikZapytania)){  
        echo '<tr>';  
        echo "<td> $wiersz[0] </td>";  
        echo "<td> $wiersz[1] </td>";  
        echo '</tr>';  
    }  
}
```

Interpretacja zmiennych (Interpolacja)

To najważniejsza różnica w codziennym kodowaniu.

Cudzysłów ("): PHP sprawdza zawartość stringa i jeśli znajdzie tam zmienną (np. \$name), automatycznie wstawi jej wartość do tekstu.

Apostrof ('): PHP traktuje wszystko dosłownie. Jeśli wpiszesz \$name, w przeglądarce zobaczysz po prostu napis "\$name".

```
$imie = "Jan";  
  
echo "Witaj, $imie!"; // Wynik: Witaj, Jan!  
echo 'Witaj, $imie!'; // Wynik: Witaj, $imie!
```

Sekwencje specjalne (Escape characters)

Cudzysłowy obsługują znacznie szerszą gamę znaków specjalnych, które pozwalają formatować tekst.

- **W cudzysłowie ("):** Możesz używać znaków takich jak `\n` (nowa linia), `\t` (tabulacja) czy `\r`. Zostaną one poprawnie zinterpretowane.
- **W apostrofie ('):** Większość tych znaków nie zadziała. `\n` zostanie wyświetlone jako tekst. Jedyne co możesz "eskejpować", to sam apostrof (`\'`) oraz backslash (`\\`).

Wydajność

Z technicznego punktu widzenia apostrofy są minimalnie szybsze. Ponieważ silnik PHP nie musi skanować tekstu w poszukiwaniu zmiennych ani analizować złożonych sekwencji specjalnych, proces ten zajmuje mniej zasobów. W nowoczesnych aplikacjach różnica ta jest jednak niemal niezauważalna, chyba że przetwarzasz gigantyczne ilości danych w pętlach.

Dobra praktyka: Składnia klamrowa

Używając cudzysłowów i chcąc uniknąć niejasności (np. gdy zmienna sąsiaduje bezpośrednio z innym tekstem), warto używać klamerek {}. To sprawia, że kod jest znacznie czytelniejszy dla programisty i dla parsera PHP.

```
$owoc = "jabłk";  
echo "Mam dwa {$owoc}a"; // Wynik: Mam dwa jabłka
```

Dodawanie rekordów

DODAWANIE REKORDÓW DO BAZY W OPARCIU O INFORMACJE
WPROWADZONE W FORMULARZU

Połączenie z bazą

Nasza aplikacja musi mieć połączenie z bazą. W przypadku kilku skryptów można napisać osobny skrypt łączący z bazą przed innymi skryptami i po wszystkich skryptach skrypt zamykający połączenie. We wszystkich pozostałych skryptach wystarczy tylko sprawdzić czy mamy połączenie.

```
1  <?php
2  |      $idPolaczenia = mysqli_connect("localhost
3  |      ?>
```

```
75  <?php
76  |      if( $idPolaczenia)
77  |          mysqli_close($idPolaczenia);
78  |      ?>
```

Skrypt – krok 1

Należy sprawdzić czy istnieje połączenie oraz czy wysłano formularz (czy w tablicy \$_POST istnieją wskazane klucze)

```
<?php
    if ($idPolaczenia && isset($_POST['nazwa'],
                                $_POST['cena'],
                                $_POST['promocja'],
                                $_POST['idDostawcy'])) {
```

Skrypt – krok 2

Dla wygody można wczytać do zmiennych wartości z tablicy \$_POST

```
$nazwa      = $_POST['nazwa'];  
$cena       = $_POST['cena'];  
$promocja   = $_POST['promocja'];  
$idDostawcy = $_POST['idDostawcy'];
```

Skrypt – krok 3

Należy sformułować i wysłać do bazy zapytanie. Formułując zapytanie warto sprawdzić czy klucz główny jest AUTO_INCREMENT i jeśli tak to najlepiej go pominąć w zapytaniu. W prezentowanym zapytaniu pole id miało tą własność i dlatego zostało pominięte przy wstawianiu.

```
$zapytanie = "INSERT INTO towary( nazwa, cena, promocja, idDostawcy)
VALUES ('{$nazwa}', '{$cena}', '{$promocja}', '{$idDostawcy}');"
$wynikZapytania = mysqli_query($idPolaczenia, $zapytanie);
```

Odświeżanie nagłówka

Aby przy każdym kliknięciu przycisku odświeżania nie dodawał się do bazy ten sam rekord należy wyczyścić zawartość zmiennej superglobalnej POST. Najlepszym sposobem jest jej ponowne zaczytanie. W tym celu wydajemy polecenie ponownego wczytania strony w trybie natychmiastowym:

```
header("location:index.php");  
exit;
```

Robimy to zaraz po dodaniu rekordu do bazy – w tym samym skrypcie. Cały skrypt na następnym slajdzie.

Dodanie do bazy - całość

```
if ($idPolaczenia && isset($_POST['nazwa'],
                                $_POST['cena'],
                                $_POST['promocja'],
                                $_POST['idDostawcy'])) {
    $nazwa      = $_POST['nazwa'];
    $cena       = $_POST['cena'];
    $promocja   = $_POST['promocja'];
    $idDostawcy = $_POST['idDostawcy'];
    $zapytanie = "INSERT INTO towary( nazwa, cena, promocja, idDostawcy)
                  VALUES ('{$nazwa}', '{$cena}', '{$promocja}', '{$idDostawcy}')";
    $wynikZapytania = mysqli_query($idPolaczenia, $zapytanie);
    header("location:index.php");
    exit;
}
```

Nagłówki w PHP

W PHP nagłówki (headers) powinny/muszą zostać wysłane **zanim** pojawi się jakikolwiek kod HTML, tekst czy echo.

Najlepszą praktyką jest umieszczenie całej logiki PHP, która może wykonywać przekierowania (obsługa formularzy, usuwanie danych), na samym początku pliku, przed tagiem `<!DOCTYPE html>`.

Nagłówki w PHP

```
1  <?php
2      $idPolaczenia = mysqli_connect("localhost","root","","sklep2");
3
4      // LOGIKA DODAWANIA - dodawanie rekordu do bazy
5      if ($idPolaczenia && isset($_POST['nazwa'], $_POST['cena'], $_POST['promocja'], $_POST['idDostawcy'])) {
6          $nazwa      = $_POST['nazwa'];
7          $cena        = $_POST['cena'];
8          $promocja     = $_POST['promocja'];
9          $idDostawcy  = $_POST['idDostawcy'];
10         $zapytanie = "INSERT INTO towary( nazwa, cena, promocja, idDostawcy)
11                        VALUES ('{$nazwa}', '{$cena}', '{$promocja}', '{$idDostawcy}')";
12         mysqli_query($idPolaczenia, $zapytanie);
13         header("location:index.php");
14         exit;
15     }
```

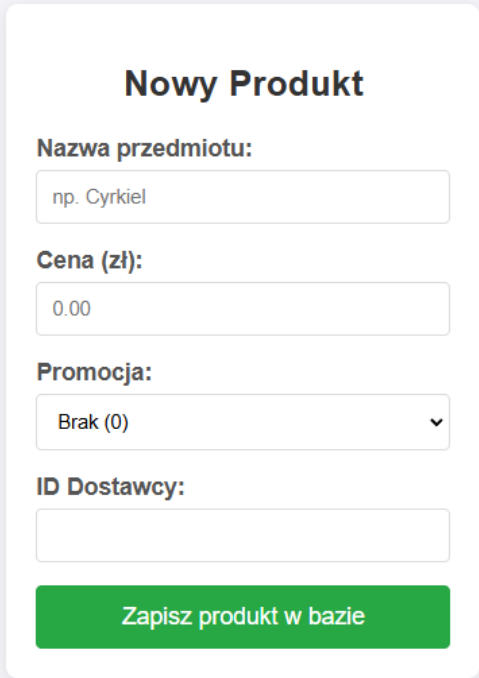

Nagłówki w PHP

Z tych samych powodów usuwanie rekordu z bazy przenosimy na górę. Cały skrypt wygląda wtedy następująco:

```
1  <?php
2      // połączenie z bazą danych
3      $idPolaczenia = mysqli_connect("localhost","root","","sklep2");
4
5      // LOGIKA DODAWANIA - dodawanie rekordu do bazy
6  >  if ($idPolaczenia && isset($_POST['nazwa'], $_POST['cena'], $_POST['promocja'], $_POST['idDostawcy'])) { ...
16     }
17     // LOGIKA USUWANIA - usuwanie rekordu z bazy
18  >  if ($idPolaczenia && isset($_GET['usun'])) { ...
24     }
25  ?>
26  <!DOCTYPE html>
27  <html lang="pl">
```

Dalsze kroki

Poprzedni skrypt został skończony, ale na ten moment użytkownik nie wie czy operacja dodania do bazy skończyła się sukcesem. Można to oczywiście oprogramować i korzystając z instrukcji warunkowej wyświetlić na ekranie odpowiedni komunikat. W podanym przykładzie zobaczymy jak można to zobaczyć obserwując liczbę elementów w bazie:



Nowy Produkt

Nazwa przedmiotu:

Cena (zł):

Promocja:

ID Dostawcy:

Zapisz produkt w bazie

W sklepie znajduje się aktualnie 27 towarów.

Dodatkowy skrypt

Skrypt wyświetla akapit z informacją o liczbie towarów w bazie, dzięki użytemu ponownemu wczytaniu strony (funkcja *header* we właściwym miejscu), liczba aktualizuje się sama :

```
<?php
    if ($idPolaczenia){
        $zapytanie = "SELECT COUNT(*) FROM towary";
        $wynikZapytania=mysqli_query($idPolaczenia,$zapytanie);
        $ile = mysqli_fetch_row($wynikZapytania);
        if ($ile)
            echo $ile[0];
    }
?>
```

HEREDOC

Pojęcie Heredoc

Definicja

- Heredoc to sposób definiowania ciągów znaków (stringów), który pozwala na zachowanie formatowania tekstu (nowe linie, wcięcia) bez konieczności używania znaków ucieczki (np. `\n`).

Zastosowanie

- Idealny do długich bloków kodu HTML, zapytań SQL lub szablonów wiadomości e-mail.

Główna zaleta

- Tekst wewnątrz Heredoc zachowuje się jak string w cudzysłowie (`""`), co oznacza, że zmienne są w nim interpretowane.

HEREDOC Operator otwarcia i identyfikator

Konstrukcja zaczyna się od trzech znaków mniejszości <<<. Bezpośrednio (bez żadnej spacji) po nich następuje identyfikator czyli nazwa.

- Nazewnictwo: Identyfikator musi spełniać te same zasady co nazwy zmiennych w PHP: może zawierać litery, cyfry i podkreślniki, ale nie może zaczynać się od cyfry.
- Standardy: Najczęściej używa się wielkich liter (np. HTML, EOD, SQL), aby blok był widoczny z daleka, ale małe litery też zadziałają.

HEREDOC – zawartość, czyli co warto w nim umieszczać

Sytuacje w których warto używać *heredoc* czyli co w nim umieszczamy:

- Duże fragmenty kodu *HTML* lub *JavaScript*. Podczas ich wklejania nie trzeba przejmować się cudzysłowami.
- Zapytania *SQL*, zwłaszcza podczas budowy zapytań SQL z wieloma liniami i zmiennymi.
- Szablony e-mail bezpośrednio w kodzie.

HEREDOC - Anatomia Linii Zamykającej

Wiersz zamykający składa się z identyfikatora oraz średnika.

Identyfikator

- Musi być dokładnie taki sam jak ten użyty na początku (wielkość liter ma znaczenie!).

Średnik

- Musi pojawić się zaraz po identyfikatorze.

HEREDOC - Interpretacja zmiennych (Interpolacja)

To najważniejsza funkcja Heredoc. Zachowuje się on jak string w cudzysłowie („”).

- Zmienne proste: Możesz wrzucić \$zmienna bezpośrednio w tekst.
- Zmienne złożone: W przypadku tablic lub obiektów, dobrą praktyką jest używanie nawiasów klamrowych {}.

Heredoc – na co uważać

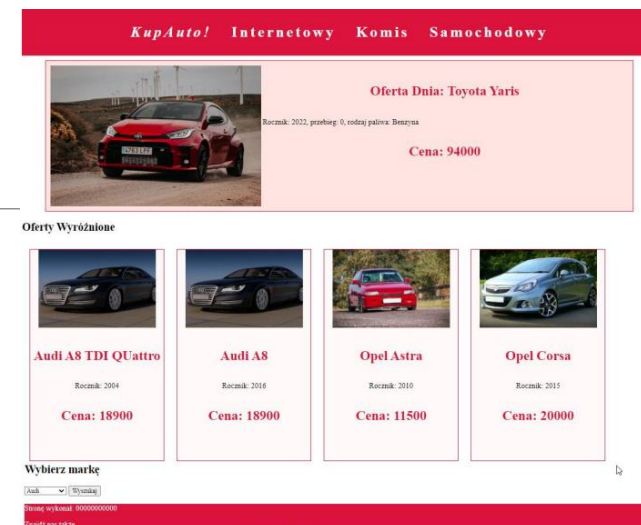
- **Zasada wyrównania:** Wcięcie znacznika zamykającego wyznacza "margines" dla całego bloku tekstu.
- **Usuwanie spacji:** PHP usuwa z każdej linii tekstu tyle białych znaków, ile znajduje się przed znacznikiem zamykającym.
- **Ograniczenie:** Treść wewnątrz Heredoc nie może być wysunięta bardziej w lewo niż znacznik zamykający. Jeśli tak zrobisz, PHP zgłosi błąd.
- **Białe znaki:** Po identyfikatorze w pierwszej linii nie może znaleźć się absolutnie nic – nawet jedna spacja czy tabulacja przed przejściem do nowej linii.

Heredoc - przykłady

Fragmenty pętli while w skryptach łączenia z bazą Zestawu nr 5 czerwiec 2024

```
echo <<<HTML_BLOCK
    
    <section>
        <h4>Oferta Dnia: Toyota $wiersz[0]</h4>
        <p>
            Rocznik: $wiersz[1],
            przebieg: $wiersz[2],
            rodzaj paliwa: $wiersz[3]
        </p>
        <h4>Cena: $wiersz[4]</h4>
    </section>
HTML_BLOCK;
```

```
echo <<<HTML_BLOCK
    <section class="oferta">
        |
        <h4> $wiersz[0] $wiersz[1]</h4>
        <p> Rocznik: $wiersz[2]</p>
        <h4> Cena: $wiersz[3]</h4>
    </section>
HTML_BLOCK;
```



Heredoc - przykłady

Zadanie z wyświetlaniem i usuwaniem rekordów:

```
while ($wierszZapytania = mysqli_fetch_row($wynikZapytania)){
    $cena = number_format($wierszZapytania[2],2)."zł";
    echo <<<BLOK_HTML1
        <tr>
            <td> $wierszZapytania[1]</td>
            <td>
                <span id="cena">
                    $cena
                </span>
            </td>
            <td> <a href="index.php">Edytuj</a> </td>
            <td> <a href="index.php?usun=$wierszZapytania[0]">Usuń</a></td>
        </tr>
    BLOK_HTML1;
}
```

Nowy Produkt

Nazwa przedmiotu:

Cena (zł):

Promocja:

ID Dostawcy:

Zapisz produkt w bazie

Lista produktów

Nazwa produktu	Cena	Zmiany	
Zeszyt 60 kartek	4.50zł	Edytuj	Usuń
Zeszyt 32 kartki	1.20zł	Edytuj	Usuń
Cykiel	12.40zł	Edytuj	Usuń
Linijka 30 cm	7.20zł	Edytuj	Usuń
Ekierka	5.50zł	Edytuj	Usuń
Linijka 50 cm	8.20zł	Edytuj	Usuń
Gumka do mazania	3.20zł	Edytuj	Usuń
Cienkopis	2.50zł	Edytuj	Usuń
Pisaki 60 szt.	55.00zł	Edytuj	Usuń
Markery 4 szt.	22.40zł	Edytuj	Usuń

Wyświetlenie ceny

Aby cena wyświetlała się z dwoma miejscami po przecinku użyto funkcji *numer_format* opisanej na następnym slajdzie oraz włożono ją w wystylizowany znacznik *span*, aby pokazywała się równo wyrównana do prawej strony. Style ustawione zostały podobnie jak stylizujemy znaczniki label (etykiety)

```
#cena{  
    display: inline-block;  
    width: 120px;  
    text-align: right;  
}
```

Formatowanie liczb

```
number_format(number, decimals, decimalpoint, separator)
```

Parameter Values

Parameter	Description
<i>number</i>	Required. The number to be formatted. If no other parameters are set, the number will be formatted without decimals and with comma (,) as the thousands separator.
<i>decimals</i>	Optional. Specifies how many decimals. If this parameter is set, the number will be formatted with a dot (.) as decimal point
<i>decimalpoint</i>	Optional. Specifies what string to use for decimal point
<i>separator</i>	Optional. Specifies what string to use for thousands separator. Only the first character of separator is used. For example, "xxx" will give the same output as "x" Note: If this parameter is given, all other parameters are required as well

Aktualny wygląd strony

Nowy Produkt

Nazwa przedmiotu:

Cena (zł):

Promocja:

Brak (0) ▾

ID Dostawcy:

Zapisz produkt w bazie

Lista produktów

Nazwa produktu	Cena	Zmiany	
Zeszyt 60 kartek	4.50zł	Edytuj	Usuń
Zeszyt 32 kartki	1.20zł	Edytuj	Usuń
Cyrkiel	12.40zł	Edytuj	Usuń
Linijka 30 cm	7.20zł	Edytuj	Usuń
Ekierka	5.50zł	Edytuj	Usuń
Linijka 50 cm	8.20zł	Edytuj	Usuń
Gumka do mazania	3.20zł	Edytuj	Usuń
Cienkopis	2.50zł	Edytuj	Usuń
Pisaki 60 szt.	55.00zł	Edytuj	Usuń
Markery 4 szt.	22.40zł	Edytuj	Usuń

W sklepie znajduje się aktualnie 10 towarów.

GET - przypomnienie

GET to jedna z metod protokołu HTTP. Służy do wysyłania żądań na serwer.

Widoczność: Dane są częścią adresu URL. Każdy może je zobaczyć w pasku przeglądarki.

Zakładki: Stronę z parametrami GET (np. wyniki.php?strona=2) można zapisać w zakładkach – parametry zawsze zostaną przesłane tak samo.

Historia: Adresy z GET zostają w historii przeglądarki.


```
<a href="index.php?usun=15">Usuń rekord</a>
```

Linki z GET

Link z parametrem GET to sposób na "wysłanie wiadomości" do serwera bezpośrednio przez adres strony.

Budowa linku w HTML:

Wartość, która ma zostać przesłana (np. ID produktu), powinno się doklejać po znaku zapytania.

Zasady tworzenia ciągu zapytania są sztywne:

? – Zaczyna listę parametrów.

klucz=wartosc – Para danych.

& – Łącznik, jeśli chcesz przesłać więcej niż jedną rzecz.

Budowa linku z get - przykład

Przykład wielu parametrów:

```
<a href="index.php?akcja=usun&id=15&potwierdzenie=tak">  
    Usuwanie  
</a>
```

W PHP tablica \$_GET będzie miała trzy elementy o kluczach odpowiednio:

- akcja
- id
- potwierdzenie

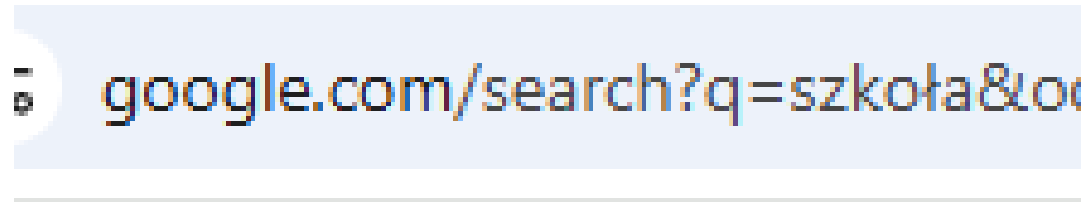
Tablica superglobalna \$_GET

W PHP dane z linku trafiają do specjalnej tablicy asocjacyjnej \$_GET. Jest ona dostępna w każdym miejscu skryptu.

```
// Adres: sklep.php?id=105&kolor=czerwony  
  
echo $_GET['id'];      // Wypisze: 105  
echo $_GET['kolor'];  // Wypisze: czerwony
```

GET w praktyce (Wyszukiwarka)

Najlepszym przykładem użycia GET jest wyszukiwarka Google. Można zauważyć, że po wpisaniu hasła, np. „szkoła”, adres zmienia się na coś w stylu:

A screenshot of a web browser's address bar. The address bar has a light blue background. On the left, there is a small icon of a globe. To the right of the icon, the text "google.com/search?q=szkoła&loc" is displayed in a dark blue font. The text is partially cut off on the right side.

google.com/search?q=szkoła&loc

Znacznik `<a>` (anchor)

Jest to podstawowy element języka HTML, który służy do tworzenia *hiperłączy*, czyli po prostu linków. Jego budowa w najprostszej postaci:

```
<a href="adres_strony1">
    Tekst widoczny na ekranie
</a>

<a href="adres_strony2">
    <!-- Obraz będący linkiem -->
    
</a>
```

Atrybuty znacznika <a>

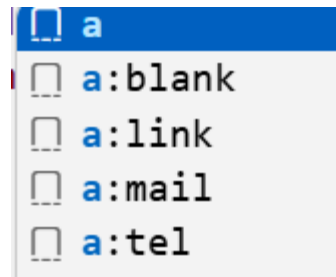
Atrybut	Co robi?	Przykład
href	Adres docelowy	href="strona.pl"
target	Gdzie otworzyć	target="_blank"
download	Wymusza pobranie	download
rel	Relacja między stronami	rel="nofollow"
title	Opis po najechaniu	title="Zobacz więcej"

Atrybuty znacznika <a>

- **download:** Informuje przeglądarkę, że zamiast otwierać plik (np. obrazek lub PDF), powinna go pobrać. Przykład: `<a href="obrazek.jpg" download>Pobierz zdjęcie</a>` Można też podać nową nazwę pliku: `download="nowa_nazwa.jpg"`.
- **rel (Relationship):** to jeden z najważniejszych elementów znacznika `<a>` z punktu widzenia SEO (pozycjonowania), bezpieczeństwa oraz dostępności. Informuje on wyszukiwarki i przeglądarki, w jakim celu łączysz się z daną stroną.
- **title:** Dodaje krótki opis, który pojawia się jako dymek (tooltip), gdy użytkownik najedzie myszką na link.
- **type:** Określa typ mediów (MIME type) dokumentu docelowego, np. `type="text/html"` lub `type="application/pdf"`. Pomaga przeglądarce zdecydować, jak obsłużyć plik.

Atrybut *href* znacznika `<a>`

Emmet Visual Studio Code ułatwia nam wybór niektórych wartości tego atrybutu:



```
<a href="mailto:"></a>
```

```
<a href="tel:+"></a>
```


Żądanie GET w linku

W protokole HTTP każde kliknięcie w standardowy link `<a>` powoduje wysłanie żądania typu GET. Jest to najpopularniejsza metoda komunikacji w Internecie, służąca do pobierania (odbierania) danych z serwera.

W metodzie GET dane przesyłane są w tzw. Query String (ciągu zapytania), który dopisuje się na końcu adresu po znaku zapytania `?`

Budowa takiego linku: `<a href="szukaj.php?produkt=rower&kolor=czerwony">Szukaj roweru</a>`

➤ `szukaj.php` - to plik lub ścieżka na serwerze, która odbierze dane.

➤ `?` - to symbol rozpoczynający listę parametrów.

➤ `produkt=rower` - to pierwsza para klucz=wartość.

➤ `&` - łącznik kolejnych parametrów.

➤ `kolor=czerwony` - to druga para klucz=wartość.

Typowe zastosowania

1. Stronicowanie (Paginacja):

```
<a href="blog.php?strona=2">Następna strona</a>
```

2. Filtrowanie produktów:

```
<a href="sklep.php?kategoria=buty&rozmiar=42">Buty rozmiar 42</a>
```

3. Wybór języka:

```
<a href="index.php?lang=pl">Wersja polska</a>
```

Usuwanie rekordu

W tworzonej aplikacji

usuwanie rekordu

realizowane jest w pętli

while podczas wyświetlania

produktów z bazy.

```
while ($wierszZapytania = mysqli_fetch_row($wynikZapytania)){
    $cena = number_format($wierszZapytania[2],2)."zł";
    echo <<<BLOK_HTML1
        <tr>
            <td> $wierszZapytania[1]</td>
            <td>
                <span id="cena">
                    $cena
                </span>
            </td>
            <td> <a href="index.php">Edytuj</a> </td>
            <td>
                <a href="index.php?usun=$wierszZapytania[0]">
                    Usun
                </a>
            </td>
        </tr>
    BLOK_HTML1;
}
```

Styl tabeli

Można popracować nad stylem, aby zwiększyć czytelność i podnieść atrakcyjność strony.

Lista produktów

Nazwa produktu	Cena	Zmiany	
Zeszyt 60 kartek	4.50zł	Edytuj	Usuń
Zeszyt 32 kartki	1.20zł	Edytuj	Usuń
Cyrkiel	12.40zł	Edytuj	Usuń
Linijka 30 cm	7.20zł	Edytuj	Usuń
Ekierka	5.50zł	Edytuj	Usuń
Linijka 50 cm	8.20zł	Edytuj	Usuń
Gumka do mazania	3.20zł	Edytuj	Usuń
Cienkopis	2.50zł	Edytuj	Usuń
Pisaki 60 szt.	55.00zł	Edytuj	Usuń
Markery 4 szt.	22.40zł	Edytuj	Usuń
a	0.02zł	Edytuj	Usuń
c	0.03zł	Edytuj	Usuń

Pseudoklasy w CSS

Są to specjalne słowa kluczowe dodawane do selektorów CSS, które pozwalają stylizować elementy nie ze względu na ich nazwę czy klasę, ale ze względu na ich stan lub pozycję w kodzie. W realizowanej aplikacji pseudoklasy zostaną wykorzystane do nadania różnych kolorów kolejnym wierszom w tabeli:

```
table tr:nth-child(2n+1){  
    background-color: #28a74644;  
}  
table tr:nth-child(2n){  
    background-color: #28a74616;  
}  
table tr:first-child{  
    background-color: #28a7468b;  
}
```

Nazwa produktu	Cena	Zmiany	
Zeszyt 60 kartek	4.50zł	Edytuj	Usuń
Zeszyt 32 kartki	1.20zł	Edytuj	Usuń
Cyrkiel	12.40zł	Edytuj	Usuń
Linijka 30 cm	7.20zł	Edytuj	Usuń
Ekierka	5.50zł	Edytuj	Usuń
Linijka 50 cm	8.20zł	Edytuj	Usuń
Gumka do mazania	3.20zł	Edytuj	Usuń
Cienkopis	2.50zł	Edytuj	Usuń
Pisaki 60 szt.	55.00zł	Edytuj	Usuń
Markery 4 szt.	22.40zł	Edytuj	Usuń
a	0.02zł	Edytuj	Usuń
c	0.03zł	Edytuj	Usuń

Pseudoklasy w CSS

Aby usunąć białe linie między komórkami tabeli należy dokonać ich kolapsacji w tabeli

```
table{  
    width: 80%;  
    margin:auto;  
    color: black;  
    border-collapse: collapse;  
}
```

Nazwa produktu	Cena	Zmiany	
Zeszyt 60 kartek	4.50zł	Edytuj	Usuń
Zeszyt 32 kartki	1.20zł	Edytuj	Usuń
Cyrkiel	12.40zł	Edytuj	Usuń
Linijka 30 cm	7.20zł	Edytuj	Usuń
Ekierka	5.50zł	Edytuj	Usuń
Linijka 50 cm	8.20zł	Edytuj	Usuń
Gumka do mazania	3.20zł	Edytuj	Usuń
Cienkopis	2.50zł	Edytuj	Usuń
Pisaki 60 szt.	55.00zł	Edytuj	Usuń
Markery 4 szt.	22.40zł	Edytuj	Usuń
a	0.02zł	Edytuj	Usuń
c	0.03zł	Edytuj	Usuń

Interaktywny styl dla co drugiego elementu typu *tr* wewnątrz tabeli

Stosowanie naprzemiennych kolorów w wierszach tabeli tzw. Zebra Striping (pasy zebry) bardzo ułatwia to czytanie dużych tabel, bo oko użytkownika nie gubi się w poziomie. Dodatkowo nałożenie `:hover` na konkretnym wierszu daje świetny sygnał wizualny, który wiersz jest aktualnie analizowany. W kodzie zobaczymy „łańcuchowanie” pseudoklas:

```
table tr:nth-child(2n):hover{
    background-color: #218837a4;
}
table tr:nth-child(2n+1):hover{
    background-color: #218837a4;
}
```

Nazwa produktu	Cena	Zmiany	
Zeszyt 60 kartek	4.50zł	Edytuj	Usuń
Zeszyt 32 kartki	1.20zł	Edytuj	Usuń
Cyrkiel	12.40zł	Edytuj	Usuń
Linijka 30 cm	7.20zł	Edytuj	Usuń
Ekierka	5.50zł	Edytuj	Usuń
Linijka 50 cm	8.20zł	Edytuj	Usuń
Gumka do mazania	3.20zł	Edytuj	Usuń
Cienkopis	2.50zł	Edytuj	Usuń
Pisaki 60 szt.	55.00zł	Edytuj	Usuń
Markery 4 szt.	22.40zł	Edytuj	Usuń
a	0.02zł	Edytuj	Usuń
c	0.03zł	Edytuj	Usuń

Zadanie

Rozbudować dotychczasową aplikację o możliwość usuwania towarów z bazy. Wystylizować tabelę.